

Step-by-Step Trace

We'll walk through the array [12, -7, -5, 70, -3, 15] with left starting at index 0 and right at index 5.

Iter	left	right	arr[left]	arr[right]	Which case?	Action
1	0	5	12	15	both non-negative	move left →1
2	1	5	-7	15	left < 0 & right ≥ 0 (correct)	left →2, right →4
3	2	4	-5	-3	left < 0 & right < 0 (both neg)	move left →3
4	3	4	70	-3	left ≥ 0 & right < 0 (swap)	swap→ [12, -7, -5, -3, 70
5	4	3	—	—	loop ends (left>right)	

Detailed Explanation

- Iteration 1**
 - left=0 (12), right=5 (15) → both are non-negative.
 - No swap. We inch left rightward to skip this positive, and right leftward to skip that positive.
- Iteration 2**
 - left=1 (-7), right=4 (15) → left is negative, right is positive.
 - Both are in correct halves, so advance both pointers: left→2, right→3.
- Iteration 3**
 - left=2 (-5), right=3 (-3) → both are negative.
 - Negatives belong on the left, so just move left→3.
- Iteration 4**
 - left=3 (70), right=3 (-3) → 70 is on the left side (wrong) and -3 is on the right (wrong).
 - Swap them. After swapping, the subarray looks like [12, -7, -5, -3, 70, 15].
 - Advance pointers past the swap: left→4, right→2.
- Termination**
 - Now left (4) > right (2), so the loop stops.
 - All negatives (-7, -5, -3) have been bubbled to the left in some order.

Illustration

Before any operations:

[12, -7, -5, 70, -3, 15] L R

After Iteration 1:

[12, -7, -5, 70, -3, 15] L R

After Iteration 2:

[12, -7, -5, 70, -3, 15] L R

After Iteration 3:

[12, -7, -5, 70, -3, 15] L R

After Iteration 4 (swap 70↔-3):

[12, -7, -5, -3, 70, 15] L R

Final (negatives on left, positives on right):

[-7, -5, -3, 70, 12, 15]

Why It Works in $O(n)$

- Each step moves at least one of the two pointers inward.
- No element is visited more than once.
- In-place swaps avoid extra memory, achieving $O(1)$ space.